

Production-Ready Project: Typical Files Checklist

A stack-agnostic reference to the files and folders commonly found in a professional production project. Exact names vary by framework, language, cloud provider, and team conventions. Optional items are included only when the project uses that capability.

1. Root / Project Configuration

- README.md — overview, setup, usage, architecture, deployment.
- .gitignore — files Git should not track.
- .gitattributes — Git behavior when needed.
- .editorconfig — shared editor conventions.
- .env.example — required environment-variable names, never real secrets.
- LICENSE — when applicable.
- CHANGELOG.md — release history when maintained.
- CONTRIBUTING.md — team/open-source contribution rules when applicable.
- SECURITY.md — security reporting instructions, especially for public repositories.
- CODE_OF_CONDUCT.md — optional, commonly used for open source.
- Project manifest such as package.json, pyproject.toml, pom.xml, go.mod, or Cargo.toml.
- Dependency lock file such as package-lock.json, pnpm-lock.yaml, yarn.lock, poetry.lock, go.sum, or Cargo.lock.

2. Source Code

- src/ — main application source.
- app/ or pages/ — routes/pages, depending on framework.
- components/ — reusable UI components.
- features/ — feature/domain modules.
- services/ — API clients and external integrations.
- lib/ or core/ — shared application infrastructure.
- utils/ or helpers/ — reusable utilities.
- hooks/ — reusable frontend hooks where applicable.
- types/ or models/ — types, schemas, and models.
- config/ — application configuration.
- constants/ — shared constants.
- middleware/ — request middleware where applicable.
- styles/ — global styles/theme files.

3. Frontend / Web Files

- public/ — static assets.
- favicon.ico and/or favicon.svg — browser icon.
- apple-touch-icon.png — optional iOS icon.
- site.webmanifest or manifest.json — optional PWA metadata.
- robots.txt — crawler rules.
- sitemap.xml — search-engine sitemap.

- `og-image.png/jpg` — Open Graph sharing image.
- 404 and 500/error pages.
- Loading, empty-state, and error-state components.

4. Backend / API

- API route/controller files.
- Request/response schemas and validators.
- Authentication and authorization modules.
- Business/domain logic.
- Database/repository/data-access layer.
- Central error handling.
- `openapi.yaml/openapi.json` — optional API documentation.
- Health endpoint such as `/health`.
- Readiness/liveness endpoints when required by the platform.

5. Database

- Database schema definition.
- `migrations/` — versioned database migrations.
- Seed scripts — optional development/test data.
- Database configuration.
- Backup/restore scripts when maintained in the repository.
- `fixtures/` — optional reusable test data.

6. Tests

- `tests/` or `__tests__/`.
- Unit tests.
- Integration tests.
- API tests.
- E2E tests in `e2e/`.
- Test configuration and setup files.
- Mocks and fixtures.
- Playwright/Cypress configuration when used.
- Coverage configuration.

7. GitHub & CI/CD

- `.github/workflows/ci.yml` — CI pipeline.
- `.github/workflows/cd.yml` — deployment pipeline when separated.
- `.github/dependabot.yml` — optional dependency updates.
- `.github/CODEOWNERS` — optional ownership/review rules.
- `.github/pull_request_template.md` — optional PR template.
- `.github/ISSUE_TEMPLATE/` — optional issue templates.
- Dockerfile — when using Docker.

- `compose.yaml/docker-compose.yml` — optional local multi-service environment.
- `.dockerignore` — Docker build exclusions.
- Reverse-proxy configuration such as `nginx.conf` or `Caddyfile` — optional.

8. Documentation

- `docs/` — project documentation.
- `ARCHITECTURE.md` — optional architecture overview.
- `API.md` — optional API guide.
- `DEPLOYMENT.md` — deployment instructions.
- `RUNBOOK.md` — operational procedures.
- `docs/adr/` — optional Architecture Decision Records.

9. Quality, Security & Configuration

- TypeScript configuration such as `tsconfig.json` when using TypeScript.
- ESLint configuration such as `eslint.config.js`.
- Prettier configuration such as `prettier.config.*`.
- Security/dependency scanner configuration.
- Pre-commit/Husky configuration when used.
- Environment-specific configuration as required by the framework.
- Secrets should be stored in a secret manager/CI platform, not committed to Git.

10. Observability & Infrastructure

- Logging configuration.
- Monitoring configuration.
- Error-tracking configuration.
- Alerting configuration.
- Metrics/tracing configuration where needed.
- Infrastructure-as-code such as Terraform, Pulumi, or CloudFormation when used.
- Kubernetes manifests when Kubernetes is used: `deployment.yaml`, `service.yaml`, `ingress.yaml`, etc.

Typical Example App Tree

```

my-project/
├── .github/
│   ├── workflows/
│   │   ├── ci.yml
│   │   └── cd.yml
│   ├── CODEOWNERS
│   └── dependabot.yml
├── public/
│   ├── favicon.ico
│   ├── favicon.svg
│   ├── robots.txt
│   ├── sitemap.xml
│   ├── site.webmanifest
│   └── og-image.png
├── src/
│   ├── app/
│   ├── components/
│   ├── features/
│   ├── services/
│   ├── lib/
│   └── hooks/

```

- types/
- utils/
- styles/
- tests/
- unit/
- integration/
- e2e/
- database/
- migrations/
- seed.*
- docs/
- ARCHITECTURE.md
- DEPLOYMENT.md
- RUNBOOK.md
- .env.example
- .gitignore
- .editorconfig
- .dockerignore
- Dockerfile
- README.md
- SECURITY.md
- LICENSE
- CHANGelog.md
- package.json
- package-lock.json
- tsconfig.json
- eslint.config.js

Files You Should NOT Commit

- Real .env files containing secrets.
- API keys, passwords, tokens, private keys, or database credentials.
- node_modules/ or equivalent dependency directories.
- Generated build output when it is produced by the build system.
- Large temporary files and local IDE files.
- Production credentials or private certificates.

Quick Checklist

- README.md
- .gitignore
- .env.example
- project manifest + lock file
- src/
- public/
- favicon
- robots.txt
- sitemap.xml
- 404/error/loading/empty states
- tests/
- database migrations
- .github/workflows/
- Dockerfile if containers are used
- lint/type-check/format configuration
- API documentation where appropriate
- health check

- docs/
- monitoring/logging/error tracking
- infrastructure-as-code when applicable